

Categorical Aggregation with Catlab.jl

Overview

This vignette demonstrates the Outbreaks.jl Catlab extension, which provides a category-theoretic framework for aggregating line list data. The key insight: aggregation decomposes into three morphisms — onset assignment, temporal binning, and decategorification (counting) — with information loss occurring *only* at the final step.

The extension provides:

- **LineListACSet**: Convert a DataFrame to a categorical representation
- $\downarrow$ (aggregate): Coarsen temporal resolution via a surjective FinFunction
- $\#$ (decategorify): Count fibers to produce an epidemic curve
- $\otimes$ (tensor product): Stratify by multiple scales simultaneously
- **fibers**: Inspect which cases fall in each bin (pre-decategorification)
- **coarsen**: Functorial composition of binning maps

Setup

```
using Outbreaks
using Catlab
using DataFrames
```

Loading data

We use the Hagelloch measles dataset (188 cases, 1861) which has onset dates and school class membership — ideal for demonstrating temporal and stratified aggregation.

```
df = measles_hagelloch_1861()
println("$ (nrow(df)) cases, $(ncol(df)) columns")
first(df, 5)
```

188 cases, 12 columns

	case_ID	infector	date_of_prodrome	date_of_rash	date_of_death	age	gender	
	Int64	String3	Date	Date	String15	Int64	String3	
1	1	45	1861-11-21	1861-11-25	NA	7	f	...
2	2	45	1861-11-23	1861-11-27	NA	6	f	...
3	3	172	1861-11-28	1861-12-02	NA	4	f	...
4	4	180	1861-11-27	1861-11-28	NA	13	m	...
5	5	45	1861-11-22	1861-11-27	NA	8	f	...

Creating a LineListACSet

The first step wraps the DataFrame in a categorical structure. This is the *instance* construction: a functor from the line list schema (a category) to Set.

```
ll = LineListACSet(df, onset=:date_of_prodrome)
```

```
LineListACSet(188 cases, onset=:date_of_prodrome)
```

Temporal aggregation: the $\downarrow$ operator

The downward arrow $\downarrow$ applies a binning morphism. Mathematically, this is a surjective `FinFunction` from cases to time bins. No information is lost yet — we know *which* cases map to each bin.

```
weekly = ll ↓ Outbreaks.Week
```

```
BucketedCases(188 cases → 8 bins, by=week)
```

The result is a `BucketedCases` — an intermediate that preserves the full fiber structure.

Decategorification: the $\#$ operator

The sharp operator $\#$ counts the cardinality of each fiber, producing the familiar epidemic curve. This is where information is irreversibly lost: we forget *which* cases are in each bin.

```
epi_curve = #(weekly)
```

	week	count
	Date	Int64
1	1861-10-28	2
2	1861-11-04	3
3	1861-11-11	7
4	1861-11-18	44
5	1861-11-25	54
6	1861-12-02	63
7	1861-12-09	14
8	1862-01-20	1

Or as a one-liner:

```
#(ll, Outbreaks.Week)
```

	week	count
	Date	Int64
1	1861-10-28	2
2	1861-11-04	3
3	1861-11-11	7
4	1861-11-18	44
5	1861-11-25	54
6	1861-12-02	63
7	1861-12-09	14
8	1862-01-20	1

Inspecting fibers

Before decategorifying, we can inspect the actual cases in each bin. This is the left Kan extension — the categorical structure that counting collapses.

```
f = fibers(weekly)
println("${length(f)} weekly bins")
println("\nCases in peak week (week 4):")
f[4]
```

8 weekly bins

Cases in peak week (week 4):

	case_ID	infector	date_of_prodrôme	date_of_rash	date_of_death	age	gender	
	Int64	String3	Date	Date	String15	Int64	String3	
1	1	45	1861-11-21	1861-11-25	NA	7	f	...
2	2	45	1861-11-23	1861-11-27	NA	6	f	...
3	5	45	1861-11-22	1861-11-27	NA	8	f	...
4	7	42	1861-11-24	1861-11-28	NA	6	m	...
5	8	45	1861-11-21	1861-11-26	NA	10	m	...
6	10	45	1861-11-21	1861-11-25	NA	7	f	...
7	12	45	1861-11-20	1861-11-25	NA	7	f	...
8	14	181	1861-11-22	1861-11-29	NA	13	f	...
9	15	45	1861-11-24	1861-11-29	NA	8	m	...
10	16	181	1861-11-21	1861-11-25	NA	15	f	...
11	17	181	1861-11-20	1861-11-25	NA	10	f	...
12	18	175	1861-11-23	1861-11-27	NA	2	f	...
13	19	181	1861-11-20	1861-11-24	NA	11	m	...
14	20	181	1861-11-22	1861-11-27	NA	10	m	...
15	21	181	1861-11-23	1861-11-29	NA	13	f	...
16	22	45	1861-11-21	1861-11-25	NA	10	f	...
17	23	45	1861-11-21	1861-11-25	NA	7	f	...
18	26	45	1861-11-22	1861-11-27	NA	7	m	...
19	31	45	1861-11-21	1861-11-24	NA	9	f	...
20	32	45	1861-11-21	1861-11-24	NA	7	m	...
21	33	45	1861-11-24	1861-11-26	NA	7	f	...
22	34	181	1861-11-21	1861-11-26	NA	11	m	...
23	39	45	1861-11-23	1861-11-26	NA	10	m	...
24	44	45	1861-11-22	1861-11-24	NA	10	m	...
...	...	...	...	...	...	...	...	...

Functorial coarsening: Week → Month

A key advantage of the categorical approach: coarsening from weekly to monthly is *functorial composition* of FinFunctions. We compose Case → Week → Month without re-accessing raw data.

```
monthly = weekly ↓ Outbreaks.Month
#(monthly)
```

	month	count
	Date	Int64
1	1861-10-01	2
2	1861-11-01	108
3	1861-12-01	77
4	1862-01-01	1

Compare: going directly to monthly gives the same result (functoriality):

```
#(ll, Outbreaks.Month)
```

	month	count
	Date	Int64
1	1861-10-01	1
2	1861-11-01	89
3	1861-12-01	97
4	1862-01-01	1

Stratified aggregation with ⊗

The tensor product ⊗ creates a product scale for joint stratification. This corresponds to aggregation in a product category.

```
# Weekly counts stratified by school class
strat = #(ll, Outbreaks.Week ⊗ Class(:class))
```

```
first(strat, 10)
```

	week	class	count
	Date	Int64	Int64
1	1861-10-28	0	1
2	1861-10-28	2	1
3	1861-11-04	0	2
4	1861-11-04	1	1
5	1861-11-11	0	3
6	1861-11-11	1	1
7	1861-11-11	2	3
8	1861-11-18	0	7
9	1861-11-18	1	27
10	1861-11-18	2	10

Multiple categorical variables can be combined:

```
# By month and class
#(ll, Outbreaks.Month @ Class(:class))
```

	month	class	count
	Date	Int64	Int64
1	1861-10-01	2	1
2	1861-11-01	0	22
3	1861-11-01	1	30
4	1861-11-01	2	37
5	1861-12-01	0	67
6	1861-12-01	2	30
7	1862-01-01	0	1

Pipe-friendly syntax

All operations compose naturally with Julia's `|>` pipe:

```
result = ll |> aggregate_by(Outbreaks.Week) |> #
first(result, 5)
```

	week	count
	Date	Int64
1	1861-10-28	2
2	1861-11-04	3
3	1861-11-11	7
4	1861-11-18	44
5	1861-11-25	54

The category theory perspective

The pipeline `LineListACSet → ↓ → #` implements the mathematical decomposition:

$$\text{EpiCurve} = |-| \circ \pi^{-1} \circ \beta$$

where:

- β : onset assignment (Case $\rightarrow$ Day, attribute morphism in the ACSet)
- π^{-1} : fiber construction (preimage of the binning `FinFunction`)
- $|-|$: cardinality (decategorification, the *only* lossy step)

The `BucketedCases` intermediate lives between π^{-1} and $|-|$, preserving the full categorical structure. This is why coarsening is functorial: we compose surjections *before* counting, so `Week → Month` is just another `FinFunction` in the chain.

Summary

Operation	Symbol	Categorical meaning	Information loss?
Wrap as ACSet	LineListACSet	Instance functor $I : \mathcal{C} \rightarrow \text{Set}$	No
Aggregate	$\downarrow$	Surjective FinFunction	No
Stratify	$\otimes$	Product category	No
Coarsen	$\downarrow$ on BucketedCases	Composition of FinFunctions	No
Count	$\#$	Cardinality $\ \text{fiber}\ $	Yes
Inspect fibers	fibers	Left Kan extension	No